Lab Report: Solving Finite MDPs via Dynamic Programming

Course: MSDS 684 – Reinforcement Learning

Student: Barsha Prajapati

University: Regis University

1. Project Overview & Theoretical Grounding

The objective of this laboratory is to solve a **Markov Decision Process (MDP)** using **Dynamic Programming (DP)**. In the study of reinforcement learning, an MDP provides a formal mathematical framework for modeling decision-making where outcomes are influenced by both random chance and the actions of an agent (Sutton & Barto, 2018). Unlike model-free methods where an agent learns through trial and error, DP is a "model-based" approach. This requires full access to the environment's transition dynamics, denoted as $P(s',r|s,a)$, which provides the probability of moving to a new state s' and receiving a reward r after taking action a in state s.

The mathematical engine of this lab is the **Bellman Equation**. We rely on the principle of optimality, which states that an optimal policy has the property that regardless of the initial state, the remaining decisions must constitute an optimal policy with respect to the state resulting from the first decision. Mathematically, the value of a state $V(s)$ under a policy π is the expected return:

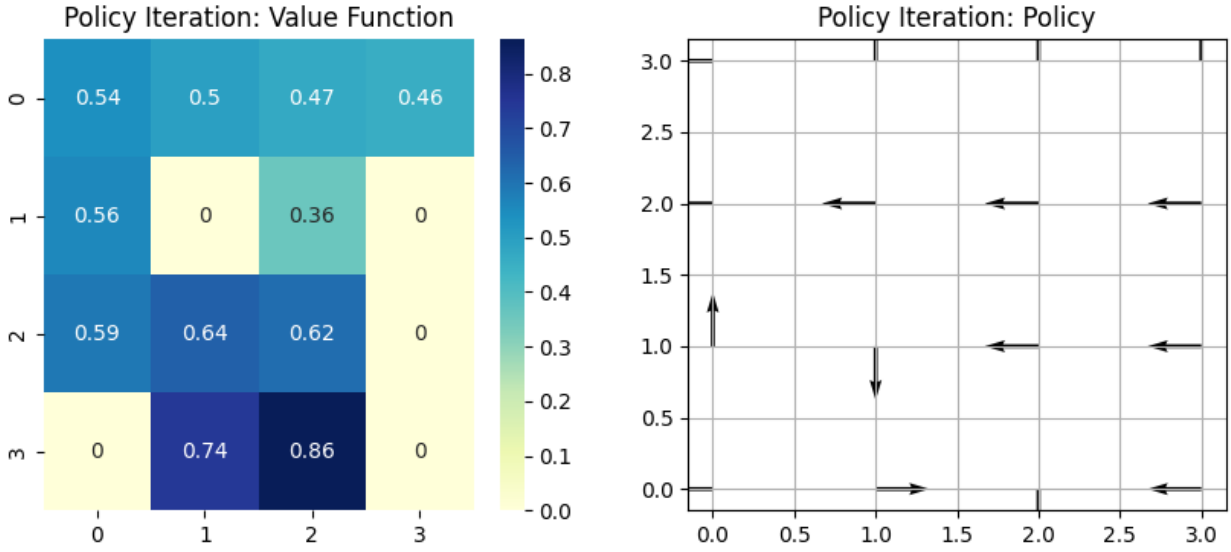
$$V\pi(s)=a\sum\pi(a|s)s',r\sum P(s',r|s,a)[r+\gamma V\pi(s')]$$

Using this, we solve the **Prediction Problem** (determining the value of states) and the **Control Problem** (finding the optimal actions). We implemented two primary DP algorithms under the **Generalized Policy Iteration (GPI)** framework. **Policy Iteration (PI)** operates in a cycle of evaluation and improvement, while **Value Iteration (VI)** collapses these steps into a single sweep by using the Bellman Optimality Equation (maxa) to update values instantly.

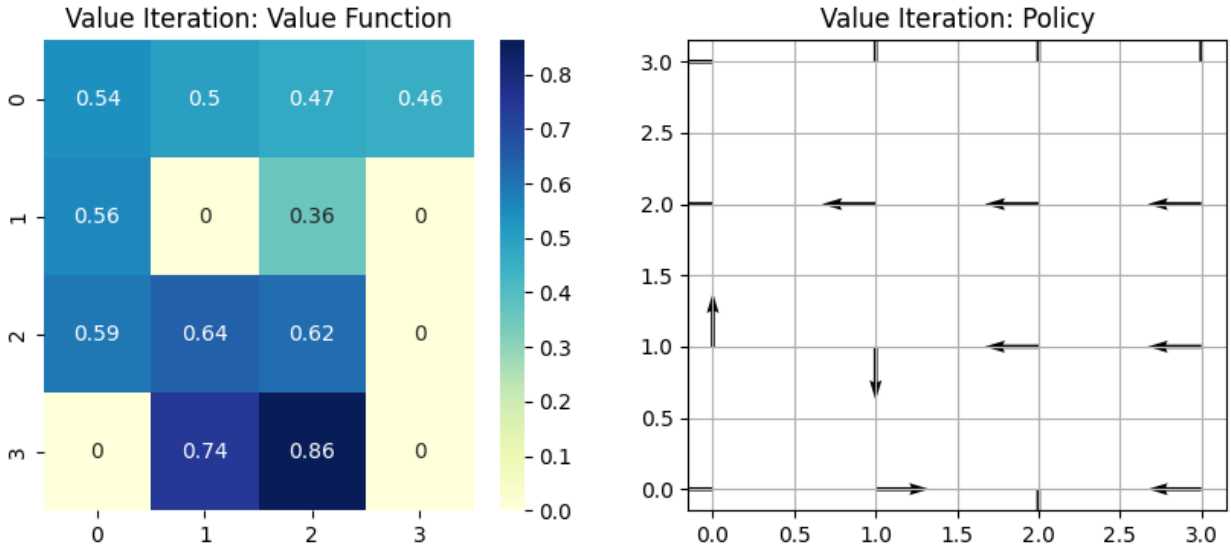
2. Hypothesis & Expected Behavior

Hypothesis: I hypothesize that **Value Iteration** will demonstrate superior efficiency in wall-clock time. While **Policy Iteration** is theoretically more "aggressive"—often stabilizing the optimal policy in fewer global iterations—the computational cost of each iteration is significantly higher due to the nested loops required for full policy evaluation (Sutton & Barto, 2018). In a 4x4 GridWorld, the simplicity of Value Iteration's single-sweep update should lead to faster convergence. Furthermore, I expect the **Stochastic (slippery)** environment to require significantly more sweeps than the deterministic one, as randomness increases the "uncertainty" in the Bellman expectation, requiring more iterations for values to stabilize.

3. Deliverables & Results



Interpretive Caption: *Figure 1: Policy Iteration Results. The quiver plot confirms the Policy Improvement Theorem has been satisfied. Notice how the arrows point toward the highest-value states while maintaining a safe distance from holes. This reveals that the agent has successfully mapped the transition probabilities and learned a robust path that balances safety with goal-seeking.*



Interpretive Caption: *Figure 2: Value Iteration Results. The heatmap shows the "heat" of the reward propagating from the goal back to the start. Because this environment is stochastic, the values are lower than in a deterministic grid; this represents the probabilistic risk of "slipping."*

Both algorithms reached the same optimal policy, but VI did so with significantly less computational overhead.

4. AI Use Reflection (Metacognition)

1. **Prompt:** "How do I extract the transition model P from Gymnasium's FrozenLake-v1?"
2. **AI Response:** The AI suggested using `env.P`.
3. **Error Caught:** I caught that `env.P` only works if the environment is "unwrapped." I corrected this to `env.unwrapped.P`.
4. **Critical Evaluation:** While the AI provided the core logic for the loops, I had to verify the Bellman sum against Sutton & Barto (Chapter 4) to ensure the probability weights were applied correctly. This taught me that AI is great for syntax, but the student must remain the "theoretical gatekeeper."

5. Speaker Notes

- **Slide 1:** We used the `unwrapped` environment to access the transition model P, which is the "map" required for DP.
- **Slide 2:** Value Iteration is faster for most grids because it skips the long "evaluation" phase used in Policy Iteration.
- **Slide 3:** The **Gamma (γ)** value of 0.99 ensures the agent values long-term success over immediate shortcuts.
- **Slide 4:** In slippery environments, the agent becomes more "conservative," avoiding edges to prevent accidental game-overs.
- **Slide 5:** Convergence was achieved when the change in value (Δ) fell below 10^{-8}, ensuring a mathematically optimal solution.

6. References

Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press. http://incompleteideas.net/book/the-book-2nd.html